DistressWatch — Judge Q&A Prep (Simple, Beginner-Friendly Answers)
SECTION A — Basic / “What is this” questions
SECTION B — Architecture / “How is it built” questions
SECTION C — The AI / Machine Learning questions (judges love this part)
SECTION D — Chatbot / Safety questions
SECTION E — Database / Security / Privacy questions
SECTION F — Phone / IVR questions
SECTION G — Frontend / UX questions
SECTION H — Testing / Quality questions
SECTION I — Limitations / honest weaknesses (say these BEFORE they ask — it builds trust)
SECTION J — Impact / “So what” questions
Quick reference — if you only remember 5 things, remember these:

DistressWatch — Judge Q&A Prep (Simple, Beginner-Friendly Answers)

Read this a few times before your presentation. Each answer is written the way you’d explain it to a friend who knows nothing about coding — you don’t need to memorize word-for-word, just understand the idea, and the words will come naturally.

SECTION A — Basic / “What is this” questions

Q1. What problem does your project solve? Crime victims and witnesses often go through a lot of emotional trauma during investigation and trial — but nobody notices until they’re already in crisis. DistressWatch checks in on them regularly (through a website or even a phone call), quietly scores how distressed they seem, and alerts a real counsellor early — before things get worse.

Q2. Who are the users of this system? Three types: (1) the Victim/Witness — the person being supported, (2) the Counsellor — a trained staff member who monitors cases and helps, (3) the Admin — who oversees the whole system across districts/states.

Q3. Why is this needed? Isn’t this already handled by counsellors manually? Human counsellors usually have too many cases to personally check on everyone every day. This system does the “noticing” part automatically and continuously, so counsellors can spend their limited time on the people who need it most, right when they need it.

Q4. Is this a chatbot app, a monitoring app, or a helpline app? All three, connected together: it’s a monitoring system (regular check-ins + a distress score), with a chat support feature, and a phone-call option, all feeding the same backend so nothing is disconnected.

Q5. What makes your project different from just a survey form? A survey form just collects answers. This system actually *understands* the answers using a trained model, calculates a live risk score, explains *why* that score was given, and automatically alerts a human when something looks dangerous — a plain form does none of that.

SECTION B — Architecture / “How is it built” questions

Q6. What is the overall architecture of your project? There are 3 main layers: (1) the **frontend** — the website people see and click on, (2) the **backend** — a Python program that receives every action and decides what to do, and (3) the **database** — where everything (cases, messages, alerts) is permanently stored. On top of the backend sit two extra brains: an **AI model** (gives a distress score) and a **safety-net checker** (catches real emergencies with simple rules). A phone system is an optional extra “door” into the same backend.

Q7. What technology/tech stack did you use? - Frontend: React (a popular JavaScript library for building interactive websites) with Tailwind CSS for styling. - Backend: Python, using a framework called FastAPI. - Database: MySQL. - AI/ML: scikit-learn (a standard Python machine-learning toolkit) — specifically Logistic Regression and Random Forest models. - Optional AI chat: Google’s Gemini API. - Phone system: Twilio (paid, real phone number) or FreeSWITCH (free, self-hosted).

Q8. Why did you choose FastAPI over other backend frameworks? FastAPI is fast, lightweight, has built-in support for validating data automatically (so bad/malicious input gets rejected before it even reaches our logic), and is one of the most widely used Python web frameworks for building APIs — it made it easy to build ~25 clearly defined “endpoints” (like doors) quickly.

Q9. Why MySQL and not something like MongoDB/Firebase? Our data is very structured and relational — a case has many events, alerts, and notes, all connected to each other by IDs. That’s exactly what a traditional relational database like MySQL is built for, and it’s a well-trusted, widely-used, free system.

Q10. Is the frontend and backend connected directly, or through an API? Through an API. The website never touches the database directly — it always sends a request to one of the backend’s ~25 endpoints (e.g. “log this chat message”), and the backend is the only thing allowed to read/write the database. This is a standard and safer way to build web apps.

Q11. How do the frontend and backend communicate in real time (for alerts)? Through a WebSocket — think of it like a phone line that stays open, instead of the website having to keep asking “anything new? anything new?” every few seconds. The moment something urgent happens, the backend pushes the alert straight to every counsellor’s screen instantly.

Q12. Does your app work without internet? The website itself works offline once loaded (all its code libraries are bundled inside it, not fetched from the internet), which is useful at a demo venue with unreliable WiFi. But actually saving/reading data still needs the backend server to be reachable — a truly full offline mode isn’t built yet.

SECTION C — The AI / Machine Learning questions (judges love this part)

Q13. How does the AI decide someone is “distressed”? It looks at two kinds of input: (1) structured answers from a check-in (mood, sleep, feeling of safety, anxiety, how much pressure the legal case is

putting on them), and (2) whatever text they type in chat. It combines both into a single score from 0 to 100, which then falls into a band: Stable, Low, Moderate, High, or Critical.

Q14. What machine learning algorithm/model did you use? Is it deep learning? No, it's not deep learning — deliberately kept simple and explainable. We used: - **Logistic Regression** with TF-IDF (a classic way to turn text into numbers) to read the free-text message and guess its risk level. - **Random Forest** (basically 140 small decision trees voting together) to combine the check-in numbers and the text's risk guess into a final score. Simple models like these are easier to explain and trust than a deep neural network — which matters a lot in a sensitive, human-safety context.

Q15. What data did you train the model on? Did you use real patient/victim data? No — and intentionally so. Using real victim data would raise serious privacy and ethical issues, especially for a hackathon prototype. We generated realistic **synthetic** (artificially created but believable) example data — about 1,800 sample cases — written to reflect how people might realistically describe different levels of distress in English, Hindi, and Hinglish.

Q16. How accurate is your AI model? We do track accuracy internally during training/testing, but we're upfront that this number is *not* a real clinical accuracy claim — because the training data is synthetic. In a real-world deployment, the model would need to be retrained and validated on real (ethically sourced, anonymized) data before its accuracy claims could be trusted for actual clinical or safety decisions.

Q17. What if the AI makes a wrong prediction — say, misses someone who's actually in danger? Two protections exist: (1) A completely separate, non-AI, rule-based safety checker independently scans every message for dangerous language and overrides the AI's opinion whenever it finds one — so the AI being "wrong" about a subtle case doesn't matter for a clear emergency. (2) A human counsellor always sees the AI's score and reasoning and can override it manually — the AI assists a human, it doesn't replace one.

Q18. Is your AI a "black box"? Can you explain its decisions? No, it's explainable by design. For every score, the system shows which factors contributed most (e.g., "poor sleep," "low safety rating") and how much each one mattered, plus a confidence level. A counsellor can always see *why* the AI gave that score, not just the number.

Q19. How does the AI handle Hindi and Hinglish (romanized Hindi), not just English? The training sentences and the safety-detection rules were written in all three: English, Hindi (Devanagari script), and Hinglish. We even had to fix a technical detail — the standard way of splitting text into words breaks on Hindi's vowel marks, so we wrote a custom rule to handle Indian scripts properly instead of relying on the English-only default.

Q20. What's the difference between the "AI model" and the "safety net" you keep mentioning? The AI model is a *soft*, learned system — it gives a probability/score based on patterns it learned, and like any ML model, it can occasionally be uncertain or wrong. The safety net is a *hard*, rule-based checker — a fixed set of patterns (like "I want to die," "he is hitting me") written by hand, not learned, so its behavior is 100% predictable and testable. For real emergencies, we deliberately trust the predictable rule-based system over the "smart" but occasionally-uncertain AI model.

Q21. Why not just use ChatGPT/an LLM for everything, including detecting emergencies? Because LLMs can be unpredictable, and in a life-and-safety situation, unpredictability is dangerous — an LLM might occasionally miss a serious phrase or respond in a way that isn't safe. So we only use a large language model (Google Gemini) for *ordinary, non-urgent* conversation, where a bit of natural variation is fine, and never let it make the actual "is this an emergency" decision.

Q22. What happens if there's no internet or the AI service (Gemini) is down? The system falls back to pre-written, safe, fixed responses instead of an AI-generated reply. The core safety detection doesn't depend on any external AI service at all — it runs entirely on our own server using simple pattern matching.

SECTION D — Chatbot / Safety questions

Q23. How does the chatbot know if a message is an emergency? It checks the message against six categories of danger: medical emergency, self-harm, violence, child safety concerns, threats/stalking, and abuse — each written in English, Hindi, and Hinglish. If it matches, the system immediately marks the case Critical, sends a pre-written safe reply, and alerts counsellors live — no AI guesswork involved for this part.

Q24. Doesn't a keyword-based system give false alarms? Like what if someone says "I am NOT suicidal"? We specifically handle that. The system checks the words right before a dangerous phrase for negation words ("not," "never," "no longer," etc.). So "I am not suicidal, just stressed" does NOT trigger an alert. But "I am not suicidal, and I want to die" still does, because it looks at each part of the sentence separately — the second half is still a real risk statement.

Q25. What if a person just replies "yes" or "haan" right after describing something urgent? The system remembers the recent conversation. A short one-word reply right after an urgent statement is treated as still being part of that urgent context, rather than resetting to "safe" just because the new message itself looks harmless.

Q26. Can the chatbot give medical or legal advice? No — it's explicitly instructed never to diagnose, never to give legal advice, and never to promise things like confidentiality or that help has already been contacted, since it can't guarantee those things. It's designed to be supportive and calming, not a replacement for a real professional.

Q27. Does the chatbot understand voice messages too? Yes — a victim can record a voice note, which gets transcribed to text (again using Google's Gemini, but strictly just for turning speech into text, not for judging emotion or risk), and then that transcript goes through the exact same safety checking as typed text.

SECTION E — Database / Security / Privacy questions

Q28. How do you protect user passwords? Passwords are never stored as plain readable text. They're run through a one-way scrambling algorithm called scrypt (with a random "salt" per user), so even someone who gets access to the raw database can't read anyone's actual password.

Q29. Can a victim see another victim's case or data? No. This is enforced on the server itself — even if someone tried to trick the website into asking for another person's case ID, the backend independently checks who's logged in and refuses. We specifically wrote automated tests to confirm this can't be bypassed.

Q30. How do you keep the data private, especially for something this sensitive? A few layers: password-protected logins with role-based access (a victim only ever sees their own data), an audit log that records every action a staff member takes on a case (for accountability), a consent system (a victim can see/control whether they're being monitored), and a data-export/delete-request feature so a victim can see or ask to remove their own data.

Q31. Is the data encrypted? Passwords are hashed (a stronger, one-way form of protection, not simple encryption). The connection between the browser and server would use HTTPS in a real deployment. For a hackathon prototype, this is the current level — a production version would add full encryption-at-rest for the database as a next step.

Q32. What happens to old/expired data? There's a privacy-request feature where a victim can request their data be deleted — currently this gets logged for staff to act on, rather than being instantly, automatically erased. That's a deliberate safety choice (so a real staff member reviews it, in case of an accidental request), but it's a manual step right now, not yet automated.

SECTION F — Phone / IVR questions

Q33. What if someone doesn't have a smartphone or internet access? They can call in on a phone instead. The system supports two options: a free, self-hosted virtual phone line (for demos) or a real, dialable phone number (via a service called Twilio). Either way, pressing a button on the keypad (like 1 for check-in, 9 for emergency) leads to the exact same backend, AI, and database as the website.

Q34. How does the phone system know which case to update? Right now, in this prototype, every phone call is linked to one fixed test case — because we don't yet have real registered phone numbers per case. In a real deployment, we'd look up the caller's phone number against a case registry to automatically identify the right person.

Q35. Is the phone call secure? Can anyone call and mess with someone's data? For the demo (free) phone line, there's a shared secret key between the phone system and the backend. For the real Twilio option, every request is cryptographically verified (checked with a signature) to make sure it genuinely came from Twilio and wasn't faked. That said, this is demo-level phone authentication, not full caller identity verification.

SECTION G — Frontend / UX questions

Q36. What does the victim actually see when they log in? A home page with a mood log, their distress trend over time, their support plan, and quick links to trusted people/resources — plus a floating chat button always available, and a prominent "I need help right now" emergency button.

Q37. What does a counsellor see? A dashboard of all their assigned cases sorted by urgency, a queue of unresolved alerts, and — when they open a specific case — a distress trend graph, an explanation of why the AI gave that score, the ability to add notes, recommend support services, or override the AI's score if they disagree (which gets logged).

Q38. Why did you build a single-page app instead of a normal multi-page website? It makes the experience feel instant — switching between screens (Home, Check-In, Chat, etc.) doesn't reload the whole page, which feels smoother and more app-like, especially important for someone who may already be stressed and doesn't want a clunky, slow interface.

SECTION H — Testing / Quality questions

Q39. How do you know this actually works and isn't just a UI mockup? We wrote real automated tests — over 40 backend tests and 23 full browser tests — that check things like: wrong passwords are truly rejected, a victim genuinely cannot see someone else's case, notes really persist after refreshing the page (so it's not just fake on-screen data), and creating multiple new cases at the same exact time doesn't create ID conflicts.

Q40. Did you find and fix any bugs while building this? Yes, a few good examples: two people creating a case at the exact same second used to sometimes get the same ID (a "race condition") — now fixed and tested; the phone check-in's mood/sleep scale used to be backwards (would have scored a person in crisis as perfectly healthy) — caught and fixed; and the standard text-splitting method broke on Hindi vowel marks — replaced with a custom method.

SECTION I — Limitations / honest weaknesses (say these BEFORE they ask — it builds trust)

Q41. What are the current limitations of your project? Be upfront and confident, not defensive: - The AI is trained on synthetic (made-up, not real) data, so its real-world accuracy is unproven — a real deployment needs proper clinical validation. - The district/state-level statistics on the Admin dashboard are demo/placeholder numbers, not computed from real data yet. - Phone calls aren't yet matched to the right person automatically. - There's no sign-up/password-reset flow yet — only a few fixed demo accounts. - Hindi/Hinglish coverage handles common phrasing, not every dialect used across India. - This is a prototype/decision-support tool, not a certified clinical or emergency-dispatch system — it's meant to route people to human help faster, not replace real crisis services.

Q42. If given more time, what would you improve next? Train the model on real, properly consented and anonymized data; connect real district-level statistics; add automatic caller-ID-to-case matching for phone calls; add proper account creation/password reset; expand language coverage; and move the frontend to a more scalable, production-grade build system.

Q43. Is this ready for real-world deployment right now? Honestly — not yet, and that's okay for a hackathon prototype. The core architecture, safety design, and workflow are solid and production-shaped, but before real deployment it would need real

validated training data, stronger infrastructure (e.g., shared rate-limiting across multiple servers), and a formal security/privacy review given how sensitive this data is.

SECTION J — Impact / "So what" questions

Q44. Who would actually use this in real life? Government victim-support programs, witness protection units, NGOs supporting trauma survivors, or hospital/legal-aid counselling teams — anywhere a small team of counsellors is responsible for many people and needs to know who to check on first.

Q45. How does this reduce the workload on human counsellors, rather than just adding more work? Instead of counsellors having to manually check in on every single case every day, the system does continuous background monitoring and surfaces only the cases that actually need attention right now — turning a "check on everyone constantly" job into a "respond to what's flagged" job.

Q46. What's the biggest innovation/USP (unique selling point) of your project? The combination of a learned AI score *and* a completely independent, predictable, rule-based safety net — most similar projects either rely entirely on an AI's judgment (risky for real emergencies) or are just static forms (miss subtle patterns over time). We do both, and we make the "why" behind every AI decision visible and overridable by a human.

Quick reference — if you only remember 5 things, remember these:

1. **Two brains, not one**: a learned AI model gives a soft distress score; a separate, simple rule-based checker independently catches real emergencies and overrides the AI. This is the single most important design decision — lead with it.
2. **Multilingual by design**: English, Hindi, and Hinglish, with proper negation handling, not an afterthought.
3. **Human stays in control**: every AI decision is explainable, and a counsellor can always override it — logged for accountability.
4. **Multiple ways in**: website or phone call, same backend, same rules.
5. **Synthetic data, honestly disclosed**: no real victim data was used, for ethical reasons — and you should say this proactively, not wait to be asked.